

B.TECH MINI PROJECT REPORT

BUDGET SHOPPING OPTIMIZER

Priority-Based Smart Shopping using 0/1 Knapsack

Submitted by

Team : [4]

Roll Number: [25MVCSMR0587,604,597,585]

Department: [CSM]

College: [MRTC]

Academic Year: [2026–2027]

1. Abstract & Introduction

1.1 Abstract

Budget Shopping Optimizer is a smart system that helps users select the best combination of products while staying within a fixed budget. The user enters a budget and a list of products with their price, value and priority. The system then uses the 0/1 Knapsack algorithm, solved with Dynamic Programming, to find the combination with the maximum total value whose total cost does not exceed the budget. The output is an optimized shopping basket with the total cost and the remaining budget.

1.2 Introduction

Most people shop with a limited budget. When several products are needed, choosing which ones to buy is difficult because every combination has a different cost and a different benefit. Choosing by guesswork can lead to overspending or to missing useful products.

The project treats shopping as an optimization problem. The budget is the limit, each product has a price (cost) and a value (benefit), and the system selects the set of products that gives the most benefit for the money available. It reduces unnecessary spending and saves the time spent comparing options.

Budget + Products + Value → Optimization → Smart Shopping Basket

2. Problem Statement, Objectives & Scope

2.1 Problem Statement

Users may need several products but have a limited budget. Manually comparing different product combinations can lead to overspending or missing useful products.

2.2 Objectives

- Stay within the user's budget.
- Select products with the maximum overall value.
- Optimize the combination of products.
- Reduce unnecessary spending.
- Calculate the total cost and the remaining budget.
- Generate an optimized shopping basket.

2.3 Scope

In scope: entering a budget; adding products with price, value and priority; selecting the best combination using 0/1 Knapsack with Dynamic Programming; showing the selected items, total cost, remaining budget and total value.

Out of scope (future work): real-time online price comparison, discount and coupon detection, AI-based recommendations, price prediction, a mobile application, nearby-store comparison and integration with online shopping platforms.

3. Features & Functional Requirements

3.1 Functional Requirements

ID	Feature	Description
FR-1	Enter budget	The user enters the total shopping budget.
FR-2	Add / select products	The user adds or selects the products needed.
FR-3	View product prices	The price of each product is entered or shown.
FR-4	Assign product value	The user gives each product a value (benefit).
FR-5	Set product priority	Each product is marked Must Have, Important or Optional.
FR-6	Optimize combination	The system finds the best combination within the budget.
FR-7	Calculate total cost	The cost of the selected products is added up.
FR-8	Show remaining budget	Budget minus total cost is displayed.
FR-9	Display shopping list	The optimized list of selected products is shown.

3.2 Priority Levels

Level	Meaning
Must Have	Items the user needs the most
Important	Items that are useful to have
Optional	Items that are nice to have

3.3 Constraint

The total price of the selected products must never exceed the budget.

4. Algorithms & Methodology

4.1 System Workflow

The system follows the steps below, from user input to the final basket.

Step	Action	Step	Action
1	User starts the system	6	Optimization engine receives the data
2	Enter budget	7	0/1 Knapsack + DP is applied
3	Select products	8	Budget constraint is checked
4	Enter / view price and value	9	Best combination is found
5	Assign priority	10	Smart shopping basket is displayed

4.2 0/1 Knapsack using Dynamic Programming

In the 0/1 Knapsack problem, every item is either taken completely (1) or left out (0). The shopping problem maps to it directly:

Shopping term	Knapsack term
Budget	Capacity
Product price	Cost (weight)
Product value	Benefit (profit)
Selected / Not selected	1 / 0

For every product the algorithm makes one choice: include it or exclude it. Dynamic Programming solves the problem by building the answer step by step. Let $dp[i][b]$ be the maximum value that can be obtained using the first i products with a budget of b . For product i with price p and value v :

- **If $p > b$** (the product is too costly): $dp[i][b] = dp[i-1][b]$
- **Otherwise** (exclude OR include): $dp[i][b] = \max(dp[i-1][b], v + dp[i-1][b-p])$

The final answer is $dp[n][\text{budget}]$. Tracing back through the table shows which products were included. The time taken is proportional to the number of products times the budget, i.e. $O(n \times \text{budget})$.

4.3 Role of Priority

Each product carries a priority (Must Have, Important, Optional), which is recorded with the product and shown in the final basket. The algorithm itself maximizes the total value assigned to the selected products within the budget.

5. Technologies & Implementation

The system is organised in layers. Each layer passes its output to the next one.

Layer	Technology	Role
Frontend	HTML + CSS + JavaScript	User enters budget, products, price, value and priority
Backend	Python	Receives the inputs and runs the logic
Optimization Engine	0/1 Knapsack + Dynamic Programming	Finds the best combination within the budget
Product Data	Items with price, value and priority	Input to the optimizer
Output	Optimized Shopping Basket	Selected items, total cost and remaining budget

5.1 Implementation Overview

The system works in three simple parts:

Part	What it does
Input	Takes the budget and, for each product, the price, value and priority.
Optimization	Runs 0/1 Knapsack with Dynamic Programming and checks that the total cost stays within the budget.
Output	Shows the selected products, the total cost, the remaining budget and the total value.

5.2 Development Tools

[Add development tools such as VS Code or GitHub here only if they were actually used.]

6. Results, Testing & Performance

6.1 Sample Optimization

Test run with a user budget of ₹1,000:

Product	Price	Value	Priority	Selected
Rice	₹300	8	Must Have	✓
Milk	₹100	5	Must Have	✓
Fruits	₹250	7	Important	✓
Snacks	₹200	4	Optional	✓
Oil	₹500	9	Important	✗

Budget	Total Cost	Remaining Budget	Total Value
₹1,000	₹850	₹150	24

The algorithm selects Rice + Milk + Fruits + Snacks, giving a total value of 24 while keeping the total cost (₹850) within the ₹1,000 budget. Oil has the highest single value (9) but costs ₹500; including it would force out other products and lower the total value.

6.2 Comparison with Other Combinations

Combination	Cost	Value	Result
Rice + Milk + Fruits + Snacks	₹850	24	Best (selected)
Rice + Milk + Oil	₹900	22	Lower value
Rice + Snacks + Oil	₹1,000	21	Lower value
Milk + Fruits + Oil	₹850	21	Lower value
Fruits + Snacks + Oil	₹950	20	Lower value

Table: Selected combination compared with other combinations that fit in ₹1,000

6.3 Testing

Test Case	Expected Result	Status
Budget validation	Total ≤ Budget	✓ Pass
Product selection	Valid combination selected	✓ Pass
Cost calculation	Correct total	✓ Pass
Optimization	Maximum value within budget	✓ Pass

6.4 Performance

Dynamic Programming checks every product against every budget level once, so the running time grows in proportion to (number of products $\times$ budget) instead of trying all 2^n combinations. This makes it practical for a normal shopping list.

7. Conclusion, Future Scope & References

7.1 Conclusion

- The system helps users shop within a fixed budget.
- It selects a high-value combination of products.
- It uses 0/1 Knapsack with Dynamic Programming.
- It reduces unnecessary spending.
- It makes shopping decisions easier.

Limitation: as it is a 0/1 model, each product can be chosen at most once, and the result depends on the values entered by the user.

7.2 Future Scope

- Real-time online price comparison
- Discount and coupon detection
- AI-based personalized recommendations
- Price prediction
- Mobile application
- Nearby-store comparison
- Integration with online shopping platforms

7.3 References

- [1] T. H. Cormen, C. E. Leiserson, R. L. Rivest and C. Stein, Introduction to Algorithms (Dynamic Programming and the 0/1 Knapsack problem), MIT Press.
- [2] H. Kellerer, U. Pferschy and D. Pisinger, Knapsack Problems, Springer, 2004.
- [3] Python Documentation, docs.python.org.
- [4] MDN Web Docs (HTML, CSS, JavaScript), developer.mozilla.org.